

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Project 2 (10%) - Library Management System (Group 6)

Name	:	IMAN ABADI BIN MOHD NIZWAN
Matric Id	:	A23CS0084

Name	:	MUHAMMAD AFIQ DANIAL BIN ROZAIDIE
Matric Id	:	A23CS0117

Name	:	MOHAMED ALIF FATHI BIN ABDUL LATIF
Matric Id	:	A23CS0112



UNIVERSITI TEKNOLOGI MALAYSIA

PRESENTATION SLIDE

PROJECT 2- Library Management System (Group 6)

Innovating Solutions

www.utm.my

Background

- A Library Management System is responsible for handling various library operations such as managing books, tracking borrowing activities, maintaining student records, and monitoring fines on students. MongoDB, as a NoSQL document-based database, provides **flexible schema design**, **high scalability**, and **efficient data handling** for this system.



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

NoSQL Data Model



COLLECTIONS LIST

- Users Collection
 - The users collection stores information about library members who are allowed to borrow books. This collection helps identify borrowers and maintain their borrowing records within the system.
- Books Collection
 - The books collection contains details of all books available in the library. This collection is used to manage the library's book catalog and support book search and loan operations.
- Loans Collection
 - The loan collection records each borrowing transaction between a user and a book. It stores references to the user and the borrowed book along with other details. This collection is for tracking active and completed book loans.
- Fines Collection
 - The fines collection stores information related to penalties imposed on users for late book returns. It has attributes that reference the related loan and user. This collection helps the library manage overdue charges and payment tracking.

COLLECTIONS LIST

```
> db.users.find()
< {
  _id: 'A23CS0001',
  name: 'Sabrina Carpenter',
  email: 'sabrina@utm.edu.my',
  contact: '0123456789',
  address: {
    street: '123 Jalan Melati',
    city: 'Muar',
    state: 'Johor'
  },
  borrowHistory: [],
  createdAt: 2024-11-20T00:00:00.000Z
}
```

- Users Collection

```
> db.books.find()
< {
  _id: 'B001',
  title: 'Database Systems Concepts',
  authors: [
    'Abraham Silberschatz',
    'Henry Korth'
  ],
  categories: [
    'Database',
    'Computer Science'
  ],
  publisher: {
    name: 'McGraw-Hill',
    year: 2021
  },
  createdAt: 2024-10-15T00:00:00.000Z
}
```

- Books Collection

```
> db.loans.find()
< {
  _id: 'L001',
  userId: 'A23CS0001',
  bookId: 'B001',
  loanDate: 2024-12-01T00:00:00.000Z,
  dueDate: 2024-12-14T00:00:00.000Z,
  returnDate: null,
  status: 'BORROWED'
}
```

- Loans Collection

```
> db.fines.find()
< {
  _id: 'F001',
  loanId: 'L001',
  userId: 'A23CS0001',
  amount: 5,
  reason: 'Late return',
  payment: {
    status: 'UNPAID',
    method: null
  },
  issuedDate: 2024-12-16T00:00:00.000Z
}
```

- Fines Collection

EMBEDDING VS REFERENCING

```
address: {  
  street: "123 Jalan Melati",  
  city: "Muar",  
  state: "Johor",  
},
```

- Embedding involves storing related data within the same document as a nested or embedded document.
- Suitable when the embedded data is tightly coupled with the main document.
- Specific to their parent document and are not shared with other collections.

- Referencing stores related data in separate collections and connects them using identifiers.
- Useful when data is shared among multiple documents
- Avoids data duplication and ensures data consistency

```
db.loans.insertOne({  
  _id: "L001",
```

```
db.fines.insertOne({  
  _id: "F001",  
  loanId: "L001",
```

DATA TYPES USED

Collection	Data Type	Attributes
Books	String	_id, title, authors, categories, publisher.name
	Number	publisher.year
	Date	createdAt
Users	String	_id, name, email, contact, address.street, address.city, address.state, borrowHistory
	Date	createdAt
Loans	String	_id, userId, bookId, status
	Date	loanDate, dueDate, returnDate
Fines	String	_id, loanId, userId, reason, payment.status, payment.method
	Number	amount
	Date	issuedDate



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Basic MongoDB Operations



Insert/Create

- insertOne()
 - This insertOne() operation can only insert one new document at one time.

```
db.users.insertOne({
```

- insertMany()
 - The insertMany() method in MongoDB adds multiple documents to a collection at once.

```
db.users.insertMany([{
```

Query/Read

- findOne()

```
// Find a user by name and retrieve specific fields using findOne()  
db.users.findOne({name: "Danial Afiq"}, {_id: 1, email: 1, contact: 1})
```

```
{  
  "_id": "A23CS0002",  
  "email": "dani@example.com",  
  "contact": "0198765432"  
}
```

The findOne() operation is a method used to retrieve a single document that matches specified criteria from a collection.

Query/Read

- find() and Projection

```
// Find all books and retrieve their titles, authors, and categories using find()  
db.books.find({}, {title: 1, authors: 1, categories: 1})
```

The find() method returns a cursor to the matching one document or more documents.

While the projection is used to specify which fields should be included in the query result. In this example, the query retrieves the all books document but only returns the _id, title, authors and categories fields instead of the entire document.

```
[  
  {  
    "_id": "B001",  
    "title": "Database Systems Concepts",  
    "authors": [  
      "Abraham Silberschatz",  
      "Henry Korth"  
    ],  
    "categories": [  
      "Database",  
      "Computer Science"  
    ]  
  },  
  {  
    "_id": "B002",  
    "title": "Introduction to Algorithms",  
    "authors": [  
      "Thomas H. Cormen",  
      "Charles E. Leiserson",  
      "Ronald L. Rivest"  
    ],  
    "categories": [  
      "Algorithms",  
      "Computer Science",  
      "Programming"  
    ]  
  },  
  {  
    "_id": "B003",  
    "title": "Advanced Mathematics",  
    "authors": [  
      "Richard G. Brown"  
    ],  
    "categories": [  
      "Mathematics",  
      "Advanced Studies"  
    ]  
  },  
]
```

Query/Read

- \$gt and \$lt

```
// Find fines with amount greater than 5.0 and less than 25.0 using $gt and $lt operators
db.fines.find({amount: {$gt: 5.0, $lt: 25.0}},
  {loanId: 1 ,amount: 1, reason: 1}).sort({amount: 1})
```

```
[
  {
    "_id": "F001",
    "loanId": "L001",
    "amount": 7,
    "reason": "Late return"
  },
  {
    "_id": "F004",
    "loanId": "L005",
    "amount": 24.5,
    "reason": "Lost book"
  }
]
```

The \$gt ("greater than") and \$lt ("less than") operators are comparison query operators used to filter documents based on a field's value relative to a specified value. They can be applied to numbers, strings, and dates.

- \$eq

```
Find fines with amount equal to RM 0.0 using $eq operator
db.fines.find({amount: {$eq: 0.0}})
```

```
Edit Document
{
  "_id": "F003",
  "loanId": "L003",
  "userId": "A23CS0003",
  "amount": 0,
  "reason": "No fine",
  "payment": {
    "status": null,
    "method": null
  },
  "issuedDate": {
    "$date": "2024-12-08T00:00:00Z"
  }
}
```

\$eq operator (equality operator) is a comparison query operator that selects documents where the value of a specified field is exactly equal to a given value.

Query/Read

- \$in

```
// Find loans with status either "BORROWED" or "PENDING" using $in operator
db.loans.find({status: {$in: ["BORROWED", "PENDING"]}}, {userId: 1, bookId: 1, status: 1})
```

```
[
  {
    Edit Document
    "_id": "L001",
    "userId": "A23CS0001",
    "bookId": "B001",
    "status": "BORROWED"
  },
  {
    Edit Document
    "_id": "L002",
    "userId": "A23CS0002",
    "bookId": "B002",
    "status": "BORROWED"
  },
  {
    Edit Document
    "_id": "L004",
    "userId": "A23CS0004",
    "bookId": null,
    "status": "PENDING"
  }
]
```

\$in operator is a comparison query operator that selects documents where the value of a field equals any value in a specified array. It is used to query for multiple possible values for a field in a single, concise query.

- \$and

```
// Find books that belong to both "Computer Science" and "Database" categories using $and operator
db.books.find({$and: [{categories: "Computer Science"}, {categories: "Database"}]}, {title: 1, authors: 1, categories: 1})
```

```
[
  {
    "_id": "B001",
    "title": "Database Systems Concepts",
    "authors": [
      "Abraham Silberschatz",
      "Henry Korth"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  },
  {
    "_id": "B006",
    "title": "Introduction to Database Systems",
    "authors": [
      "Abraham Silberschatz",
      "Henry F. Korth",
      "S. Sudarshan"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  }
]
```

\$and operator performs a logical AND operation on an array of one or more expressions and selects only those documents that satisfy all the specified conditions.

Query/Read

- \$or

```
// Find books that are either in "Programming" category or published in the year 2020 using $or operator
db.books.find({$or: [{categories: "Programming"}, {"publisher.year": 2020}],
  {title: 1, authors: 1, categories: 1, publisher: 1})
```

```
[
  {
    "_id": "B002",
    "title": "Introduction to Algorithms",
    "authors": [
      "Thomas H. Cormen",
      "Charles E. Leiserson",
      "Ronald L. Rivest",
      "Clifford Stein"
    ],
    "categories": [
      "Algorithms",
      "Computer Science",
      "Programming"
    ],
    "publisher": {
      "name": "MIT Press",
      "year": 2022
    }
  },
  {
    "_id": "B003",
    "title": "Advanced Mathematics",
    "authors": [
      "Richard G. Brown"
    ],
    "categories": [
      "Mathematics",
      "Advanced Studies"
    ],
    "publisher": {
      "name": "Springer",
      "year": 2020
    }
  }
]
```

\$or operator performs a logical OR operation on an array of two or more expressions and selects the documents that satisfy at least one of the specified conditions.

- Array Queries

```
// Find books authored by "Abraham Silberschatz" in array query
db.books.find({authors: "Abraham Silberschatz"}, {title: 1, authors: 1, categories: 1})
```

```
[
  {
    "_id": "B001",
    "title": "Database Systems Concepts",
    "authors": [
      "Abraham Silberschatz",
      "Henry Korth"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  },
  {
    "_id": "B006",
    "title": "Introduction to Database Systems",
    "authors": [
      "Abraham Silberschatz",
      "Henry F. Korth",
      "S. Sudarshan"
    ],
    "categories": [
      "Database",
      "Computer Science"
    ]
  }
]
```

Array query operations are used to search and retrieve documents where the values inside an array field match certain specified conditions.

Update

- \$set

```
// Update loan with _id "L004" to change bookId, loanDate, dueDate, and status using $set
db.loans.updateOne({_id: "L004"},
  {$set: {
    bookId: "B004",
    loanDate: new Date("2024-12-12"),
    dueDate: new Date("2024-12-26"),
    status: "BORROWED"
  }}
)
```

\$set operator is a powerful update operator used to add new fields to a document or modify the value of existing fields without overwriting the entire document. It is the primary way to perform partial updates.

- \$inc

```
// Increase fine amount by 2.0 for all fines with payment status "UNPAID"
db.fines.updateMany({"payment.status": "UNPAID"},
  {$inc: {amount: 2.0}}
)
```

\$inc operator is used to atomically increment or decrement the value of a field by a specified amount.

Update

- \$push

```
// Add a bookId to the borrowHistory array of user with _id
db.users.updateOne({_id: "A23CS0004"},
|   {$push: {borrowHistory: "B005"}}
| )
```

\$push operator appends a specified value to an array field within a document. If the field does not exist, \$push will create it as a new array field.

- \$pull

```
// Remove "Clifford Stein" from the authors array of the book titled
db.books.updateOne({title: "Introduction to Algorithms"},
|   {$pull: {authors: "Clifford Stein"}}
| )
```

\$pull operator is used to remove all instances of a specified value or values that match a certain condition from an existing array within a document.

Delete

- deleteOne()

```
// Delete the fine with _id "F003"  
db.fines.deleteOne({_id: "F003"})
```

deleteOne() operation removes at most a single document from a collection that matches the specified filter. If multiple documents match the criteria, only the first one encountered is deleted.

- deleteMany()

```
// Delete all fines with payment status "PAID"  
db.fines.deleteMany({"payment.status": "PAID"})
```

The deleteMany() operation in MongoDB is used to remove all documents from a collection that match a specified filter.

Advanced Operations

Indexing

- Single Field Index
 - A single-field index on name in users helps quickly retrieve users in alphabetical order as the index already maintains sorted data for efficient sorting operations.

```
db.users.createIndex({ name: 1 })
```

- Compound Index
 - The books collection contains details of all books available in the library. This collection is used to manage the library's book catalog and support book search and loan operations.

```
db.fines.createIndex({ userId: 1, "payment.status": 1 })
```

Aggregation

Total number of books

```
db.books.aggregate([ { $count: "totalBooks" } ])
```

```
[
  {
    "totalBooks": 6
  }
]
```

Counts the total number of documents in the books collection.

Total unpaid fines per user

```
db.fines.aggregate([ { $match: { "payment.status": "UNPAID" } },
  { $group: { _id: "$userId", totalFine: { $sum: "$amount" } } },
  { $sort: { totalFine: -1 } } ])
```

```
[
  {
    "_id": "A23CS0004",
    "totalFine": 24.5
  },
  {
    "_id": "A23CS0001",
    "totalFine": 7
  }
]
```

Filters fines to only include unpaid status, groups by userId, calculates the sum of fine amounts and sorts results in descending order by total fine amount.

Aggregation

Total books per category

```
db.books.aggregate([
  { $unwind: "$categories" },
  { $group: { _id: "$categories", totalBooks: { $sum: 1 } } },
  { $sort: { _id: 1 } }])
```

```
[
  {
    "_id": "Advanced Studies",
    "totalBooks": 1
  },
  {
    "_id": "Algorithms",
    "totalBooks": 1
  },
  {
    "_id": "Computer Science",
    "totalBooks": 4
  },
  {
    "_id": "Database",
    "totalBooks": 2
  },
  {
    "_id": "Mathematics",
    "totalBooks": 1
  }
]
```

Unwinds the categories array to create a document for each category, groups by category and sums the total number of books for each category.

Top 3 users with the highest number of books borrowed

```
db.loans.aggregate([
  { $group: { _id: "$bookId", totalBorrows: { $sum: 1 } } },
  { $sort: { totalBorrows: -1 } },
  { $limit: 3 }])
```

```
[
  {
    "_id": "A23CS0004",
    "totalBorrows": 2
  },
  {
    "_id": "A23CS0003",
    "totalBorrows": 1
  },
  {
    "_id": "A23CS0001",
    "totalBorrows": 1
  }
]
```

Groups loans by userId, counts the number of times each user has borrowed a book, and sorts results in descending order by the count.

Team Contribution

Team Role - Afiq Danial

- I contribute to CRUD implementation like insertion, query, update and also deletion.
- I suggest the real world domain which is Library Management System.
- Make a background and objectives of the report

Team Role - Iman Abadi

- Provide explanation on query performance speed from applying indexing
- Did research on NoSQL's security and limitations, covering basic access control, injection risks and its limitations.
- Concluded the whole report, summarizing what was learned and suggesting improvement.

Team Role - Alif Fathi

- Contributes to NoSQL data model part in reporting.
- Contributes to advanced operations which are index creation, aggregation pipeline and sorting demonstrations.

THANK YOU



univteknologimalaysia



utm.my

utmofficial